

家庭健康 Agent

Health Agent Harness 设计文档

v0.1 - 面向家庭长期使用的健康 Agent 运行外壳设计



图 1：Health Agent Harness 总体架构

文档定位：本设计文档用于沉淀 Health Agent Harness 的工程边界、模块职责、调用链路、数据安全规则和 V0 落地范围。它不是产品宣传文案，而是后续交给 Codex / 工程实现的技术基准。

生成时间：2026-07-02 10:07

目录

1. 1. Harness 总体定位与系统位置
2. 2. 总体架构与请求生命周期
3. 3. Harness 的三条强制链路
4. 4. runtime.py - RuntimeContext 设计
5. 5. permissions.py - 家庭权限与访问控制
6. 6. tool_registry.py - 工具注册与调用管控
7. 7. safety.py - 医疗安全边界与输出审查
8. 8. tracing.py - 执行日志与可追溯性
9. 9. memory.py - 记忆边界与长期摘要
10. 10. errors.py - 错误处理、降级与失败恢复
11. 11. guards.py - 系统硬规则
12. 12. AgentService / Runner / ToolCaller 接口骨架
13. 13. Workflow 集成关系
14. 14. V0 实施清单与测试清单
15. 15. 下一步设计建议

1. Harness 总体定位与系统位置

Health Agent Harness 是家庭健康 Agent 的运行外壳。它不替代 LangGraph，不替代数据库 service，也不直接生成医学结论。它负责让所有 Agent 工作流在真实健康产品中安全、稳定、可追踪地运行。

问题	Harness 给出的工程解法
当前用户是谁？	runtime.py 创建 RuntimeContext，身份来自登录态或系统任务白名单。
能不能看家人数据？	permissions.py 根据 family_members 与 member_share_permissions 检查。
当前 workflow 能不能调用这个 tool？	tool_registry.py 根据 ToolSpec、workflow 白名单、LLM exposure 与写入确认校验。
AI 能不能这样回答？	safety.py 检测诊断、用药、保证安全、夸大风险等表达。
以后怎么排查问题？	tracing.py 记录请求、工具调用、安全审查与错误。
Agent 记忆放什么？	memory.py 只存偏好、关注重点、会话摘要；健康事实进业务数据库。
失败时用户看到什么？	errors.py 将内部异常转成安全、可理解、可恢复的提示。
哪些事绝对不能做？	guards.py 执行硬红线：无目标不查、无确认不写、无权限不查等。

一句话定版：LangGraph 让 Agent 会做事；Harness 让 Agent 安全、稳定、可追踪、可维护地做事。

1.1 Harness 不是什么

- 不是 LangGraph 本身：LangGraph 负责流程编排，Harness 负责运行约束。
- 不是 Agent Tools：Tools 是能力，Harness 决定这些能力什么时候、由谁、以何种权限被调用。
- 不是数据库 CRUD 层：业务 service 才直接查库写库。
- 不是 Prompt 集合：它包含权限、安全、追踪、错误、记忆等工程模块。
- 不是医学诊断引擎：它只做安全边界与输出审查。

2. 总体架构与请求生命周期



图 2：Harness 在系统中的总体位置

标准层级：前端页面 / 用户操作 -> FastAPI API -> Agent Service -> Health Agent Harness -> LangGraph Workflows -> Agent Tools -> Business Services -> PostgreSQL / 文件存储。

一次 Agent 请求的标准生命周期



图 3：一次 Agent 请求的生命周期

每一次 Agent 请求必须先创建 RuntimeContext，再启动 Trace，然后将 memory_context、runtime、trace 注入 LangGraph State。workflow 内部所有 tool 调用必须先经过 ToolRegistry、Permission、Guard。回答返回前必须经过 Safety。

Harness 模块依赖关系

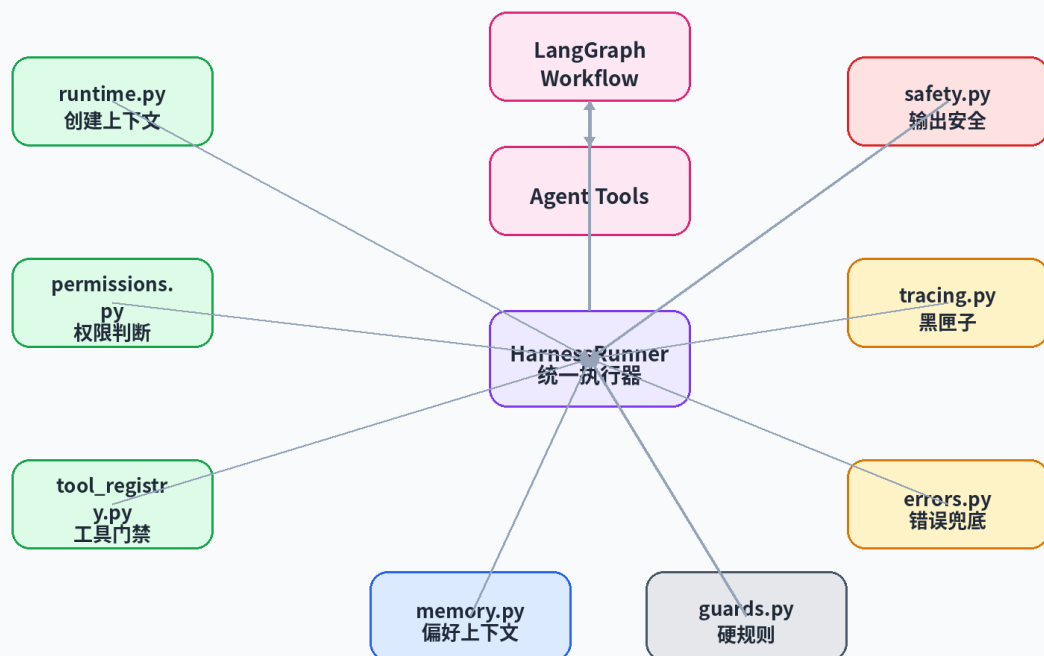


图 4: Harness 模块依赖关系

3. Harness 的三条强制链路

3.1 查询链路

```
RuntimeContext
-> Resolve Target Member
-> Permission Check
-> Tool Registry Check
-> Read Tool
-> Compose Answer
-> Safety Check
-> Trace End
-> Response
```

适用场景：问血压、问日报、问症状、问体检事件、问提醒、问资料元信息。

3.2 写入链路

写入链路：草稿 - 确认 - 入库

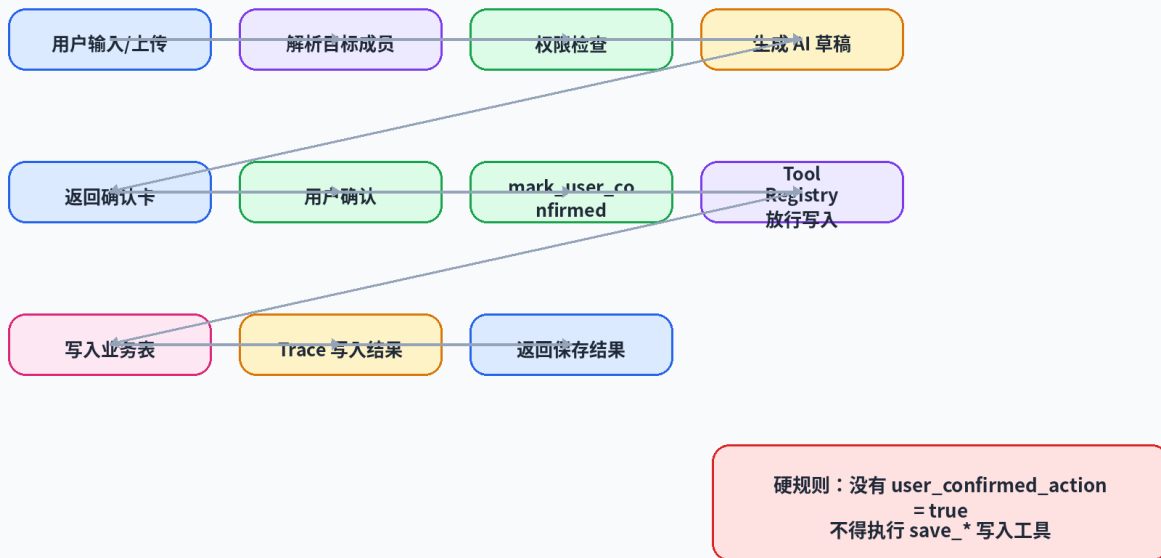


图 5: 写入链路必须经过用户确认

适用场景：健康随手记、上传资料转 medical_events、确认 AI 提取结果。核心红线是：AI 只生成草稿，用户确认后才能入库。

3.3 生成链路

```
RuntimeContext
-> Collect Data
```

- > Rule Analysis
- > LLM Compose
- > Safety Check
- > Save / Return
- > Trace

适用场景：daily_report_workflow、doctor_visit_summary_workflow。原则是：规则算事实，LLM 写表达，Safety 做边界。

4. runtime.py - RuntimeContext 设计

runtime.py 是 Harness 的地基。它回答：这次 Agent 执行是谁发起的、在哪个家庭、目标成员是谁、当前 workflow 是什么、是否用户主动触发、是否允许写入。

字段	类型/示例	设计说明
request_id	str / uuid	每次请求唯一 ID，用于 trace、tool call、error 关联。
session_id	Optional[str]	AI 管家连续对话 ID，V0 可记录但不做复杂上下文。
current_user_id	str	当前登录用户，必须来自后端登录态，不能由 LLM 或前端任意覆盖。
current_family_id	Optional[str]	当前家庭上下文；访问家人数据时必需。
workflow_name	Literal	chat_workflow / free_text_record_workflow / document_extract_workflow / daily_report_workflow / doctor_visit_summary_workflow。
trigger_type	Literal	user_chat / free_text_record / document_upload / manual_report_generate / scheduled_report_generate / doctor_visit_summary / system_job。
raw_input	Optional[str]	用户原始输入，上传/定时任务可能为空。
source_page	Optional[str]	dashboard / agent_chat / member_profile / document_page 等，帮助减少追问。
target_user_id	Optional[str]	目标成员真实 user_id，必须由后端解析或页面上上下文提供，LLM 不得自由指定。
family_member_id	Optional[str]	目标用户在当前家庭中的成员关系 ID。
target_display_name	Optional[str]	展示名，如爸爸、妈妈、本人。只用于展示，不用于权限判断。
is_system_trigger	bool	定时日报等系统任务为 true。
user_confirmed_action	bool	写入类工具必须为 true，否则 ToolRegistry / Guards 拦截。
created_at	datetime	上下文创建时间。

4.1 RuntimeManager 接口

```
class RuntimeManager:
    def create_context(...): ...
    def with_target_member(runtime, target_user_id, family_member_id, target_display_name): ...
    def mark_user_confirmed(runtime): ...
    def validate_context(runtime): ...
```

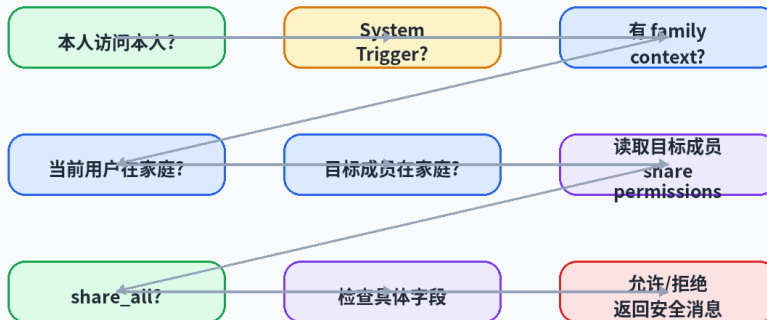
4.2 Runtime 硬规则

- 没有 RuntimeContext, 不允许执行任何 workflow。
- 没有 current_user_id, 不允许执行用户请求。
- LLM 不允许设置 current_user_id。
- LLM 不允许自由设置 target_user_id。
- 查询个人健康数据前必须有 target_user_id。
- 查询家人数据前必须有 current_family_id。
- 写入健康档案前必须 user_confirmed_action = true。
- system trigger 只能执行白名单 workflow。

5. permissions.py - 家庭权限与访问控制

permissions.py 负责判断：当前用户能不能对目标用户的某类健康数据做某个动作。它是家庭健康 Agent 的安全核心。

Permission 权限检查流程



失败原则：只说“没有权限”，不透露目标成员是否存在相关数据。

图 6：权限检查流程

5.1 permission_type 与 action

permission_type	含义	敏感等级
profile	基础健康档案	中
metrics	指标，包括睡眠、步数、心率、血压、体重等	中-高
reports	每日健康简报	中
symptoms	健康随手记 / 症状记录	高
medical_events	体检、手术、用药、过敏、慢病等正式健康事件	高
documents	原始健康资料，如 PDF、图片、病例原件	极高
alerts	关注提醒、复查提醒、资料确认提醒	中
memory_summary	Agent 长期摘要	中
doctor_visit_summary	就医摘要生成	高

action	含义	V0 优先级
view	查看数据	必做
create	创建记录	必做
generate	生成日报 / 就医摘要	必做
update	修改记录	预留
delete	删除记录	预留
export	导出 PDF / Markdown / 资料包	预留

5.2 核心判断逻辑

- 16. current_user_id == target_user_id 时，本人访问本人数据默认允许。
- 17. system trigger 单独走系统任务白名单，不继承普通用户权限。
- 18. 访问家人数据必须有 current_family_id。
- 19. 当前用户必须是该家庭 active 成员。
- 20. 目标用户必须是该家庭 active 成员。
- 21. 读取目标用户在该 family 下的 member_share_permissions。
- 22. share_all = true 时 V0 允许大部分 view/create/generate，但 documents 仍保留独立检查。
- 23. share_all = false 时检查具体权限字段。
- 24. 权限失败时返回 safe_message，不透露数据是否存在。

5.3 PermissionResult

```
class PermissionResult(BaseModel):
    allowed: bool
    permission_type: str
    action: str
    reason: str | None = None
    visibility_scope: str | None = None
    safe_message: str
```

safe_message 可以直接展示给用户；reason 只用于 trace 和调试。权限不足时禁止说“找到了资料但你没权限”，只能说“你目前没有权限查看这类数据”。

6. tool_registry.py - 工具注册与调用管控

tool_registry.py 是 Agent Tools 的门禁系统。它控制哪些工具可用、哪些 workflow 能用、哪些能暴露给 LLM、哪些必须用户确认、哪些会写库。

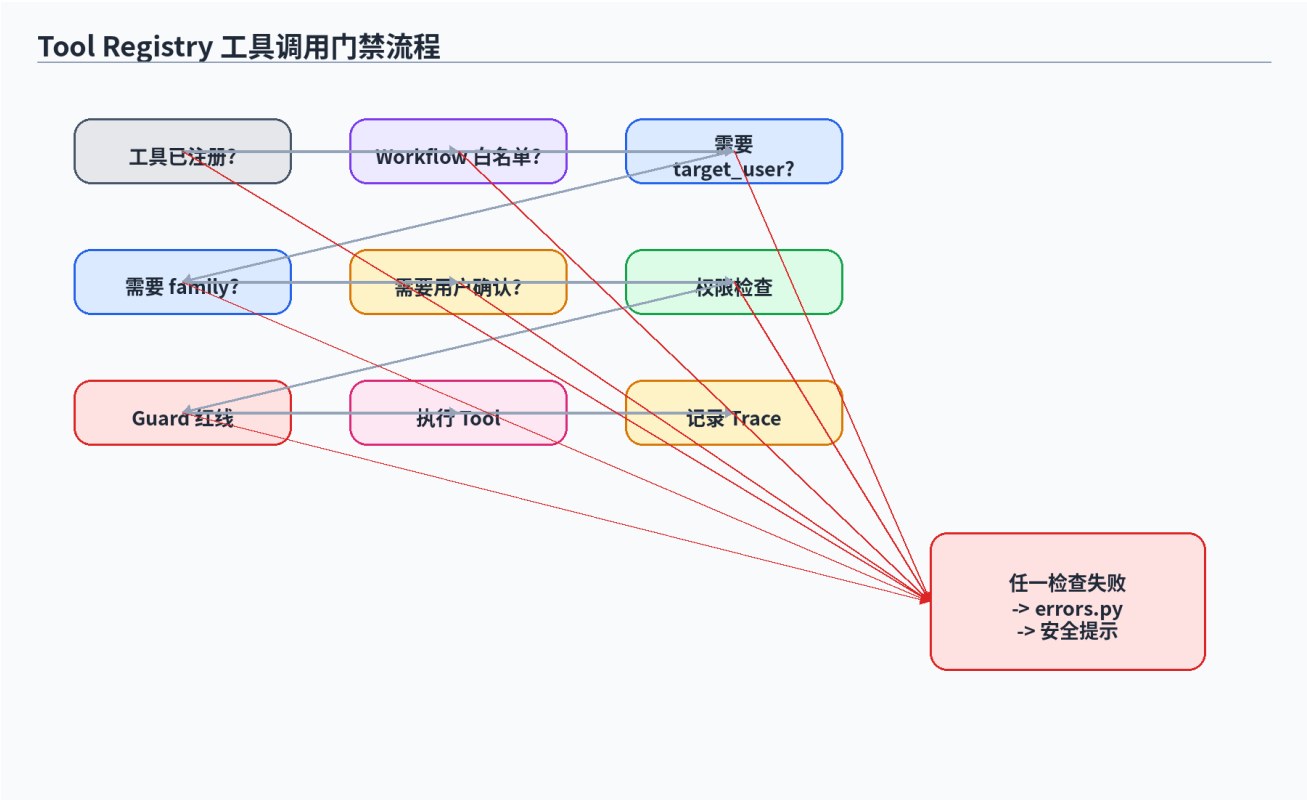


图 7: Tool Registry 工具调用门禁流程

6.1 工具分类

类别	典型工具	调用边界
前置控制类	resolve_family_member, check_member_permission	workflow 固定节点调用；不让 LLM 随意调用。
查询类	get_recent_metrics, get_recent_blood_pressure, get_symptom_records, get_medical_events, get_medical_documents, get_latest_daily_report, get_active_alerts	只读；可在受控条件下暴露给 LLM；必须有 target_user_id 和权限。
草稿生成类	create_symptom_record_draft, extract_document_summary	AI 生成草稿；不直接入库；特定 workflow 固定节点。
写入类	save_symptom_record_from_draft, save_medical_event_from_document, generate_daily_report	会写库；不得暴露给 LLM；必须用户确认或系统白名单。

6.2 ToolSpec

```
class ToolSpec(BaseModel):
    name: str
```

```

description: str
access_mode: Literal["read", "write", "draft", "control"]
risk_level: Literal["low", "medium", "high", "critical"]
exposure: Literal["llm_allowed", "workflow_only", "internal_only"]
allowed_workflows: list[str]
required_permissions: list[tuple[str, str]]
requires_target_user: bool = True
requires_family_context: bool = False
requires_user_confirmation: bool = False
can_return_sensitive_data: bool = False
can_write_database: bool = False
can_read_documents: bool = False
timeout_seconds: int = 30

```

6.3 LLM 可见工具 V0 白名单

- get_recent_metrics
- get_recent_blood_pressure
- get_symptom_records
- get_medical_events
- get_medical_documents - 仅返回元信息和 AI 摘要，不返回真实文件路径
- get_latest_daily_report
- get_active_alerts

LLM 可以决定查哪类指标、查多少天，但不能决定查谁、有没有权限、是否保存、是否读取真实文件。

7. safety.py - 医疗安全边界与输出审查

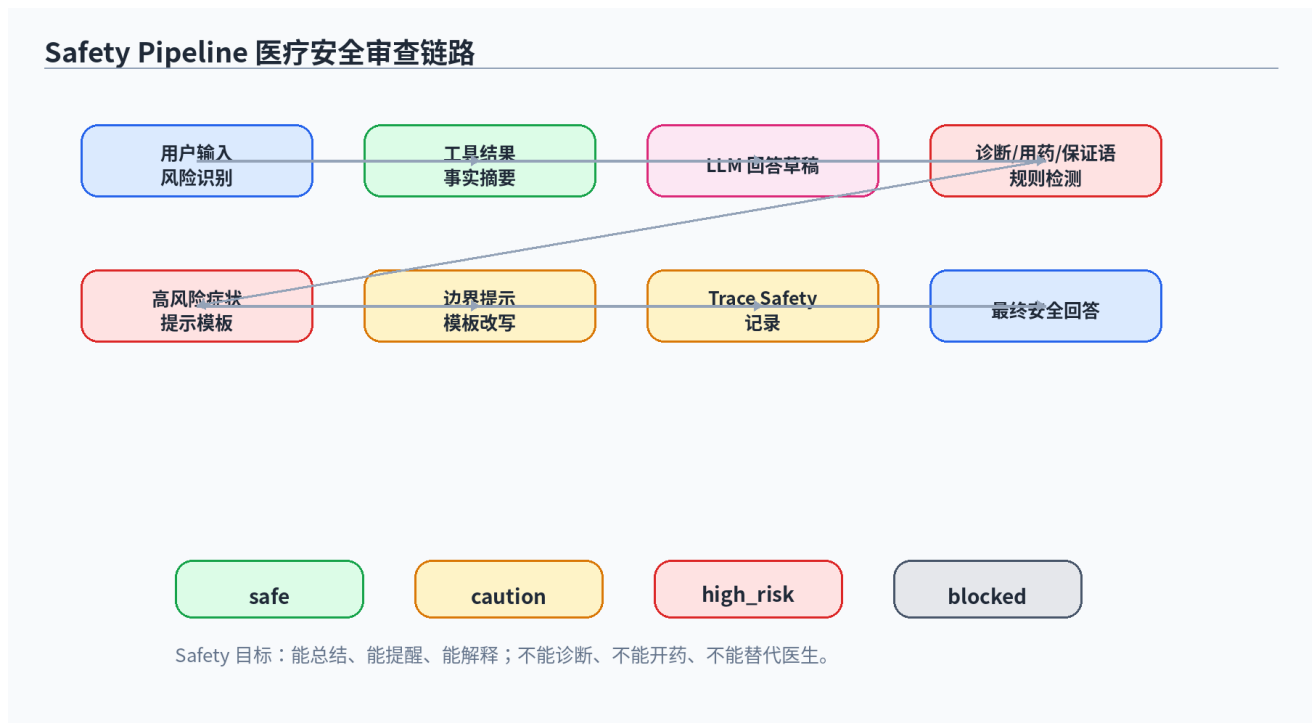


图 8: Safety Pipeline 医疗安全审查链路

safety.py 管的是“能不能这样说”。它允许 Agent 总结、提醒、解释，但不允许诊断、开药、停药或替代医生。

7.1 安全等级

等级	含义	处理方式
safe	普通健康总结，无明显风险	可直接返回，必要时保留简短边界。
caution	涉及指标变化、症状或健康建议	加医疗边界，避免确定性表达。
high_risk	胸痛、呼吸困难、昏厥、肢体无力等高风险症状	补充及时就医或急救提示。
blocked	用户要求诊断、开药、停药、替代医生判断	拒绝直接满足，转为整理资料、建议咨询医生。

7.2 高风险症状关键词 v0

胸痛、持续胸闷、呼吸困难、喘不上气、昏厥、晕倒、意识不清、说话不清、一侧肢体无力、肢体麻木、剧烈头痛、突发剧痛、大量出血、严重过敏、喉咙肿胀、持续高热、抽搐。

7.3 禁止表达

禁止类型	禁止示例	安全改写方向
诊断类	你得了高血压 / 这是心脏病	根据当前记录，相关指标值得关注，但不能据此做疾病诊断。
用药类	应该吃某某药 / 加大药量 / 停药	用药和剂量调整需要医生根据完整情况判断。
保证安全类	没事 / 肯定没问题 / 不用去医院	系统内暂无明确记录不代表现实一定没有问题。症状持续或加重请咨询医生。
夸大风险类	很危险 / 马上会出事	这个情况值得重视，尤其持续或加重时建议及时就医。
无记录当无问题	你没有过敏史 / 你没有头晕问题	我暂时没有在当前档案中找到相关记录，不代表现实中一定没有。

8. tracing.py - 执行日志与可追溯性

tracing.py 是健康 Agent 的黑匣子。它不参与决策，但记录关键过程，帮助解释“为什么这么回答”、排查“为什么没查到”、审计“谁看了谁的数据”。

8.1 推荐数据表

表名	用途	关键字段
agent_traces	一次 Agent 请求	request_id, workflow_name, trigger_type, current_user_id, target_user_id, status, duration_ms, error_code
agent_tool_calls	工具调用记录	request_id, tool_name, access_mode, risk_level, permission_result, input_summary, output_summary, status
agent_safety_checks	安全审查记录	request_id, safety_level, safety_flags,

		was_rewritten, blocked_reason, revised_answer_summary
--	--	---

8.2 不应写入日志的内容

- 完整体检报告原文、病例全文、完整 OCR 文本。
- 真实文件路径、身份证号、完整手机号、家庭详细住址。
- 完整 LLM prompt 和包含大量隐私的长聊天原文。
- 未经脱敏的医生报告全文或用户健康描述长文本。

Tracing 记录的是摘要、数量、类型、时间范围、状态和错误原因，而不是敏感原文。

9. memory.py - 记忆边界与长期摘要

Memory 设计必须克制。健康事实进业务数据库，Agent Memory 只存偏好、关注重点、会话摘要和可追溯健康摘要索引。

可以存	不能存
用户希望日报简短温和	未经确认的疾病诊断
用户近期经常关注爸爸血压	AI 推断出的病因或风险等级
本轮对话想整理爸爸血压用于就医	用药、停药、剂量建议
爸爸近期摘要涉及血压关注，详情以结构化记录为准	报告全文、病例全文、原始文件路径
默认就医摘要用最近 30 天	“系统没记录所以没有问题”的结论

9.1 agent_memories 表

字段	说明
id	memory 记录 ID
user_id	这条记忆属于哪个用户的 Agent 体验
family_id	与家庭上下文相关时记录
target_user_id	如果指向家庭成员则记录
memory_type	user_preference / attention_focus / conversation_summary / health_summary_index / workflow_preference
content	摘要化文本
source	user_feedback / interaction_summary / system_summary
source_entity_type / source_entity_id	可追溯来源
confidence_level	high / medium / low
visibility	private / family_context
status	active / archived / deleted
expires_at	短期记忆可过期

V0 建议只启用 user_preference 和 attention_focus；health_summary_index 先设计但暂不自动写入。

10. errors.py - 错误处理、降级与失败恢复

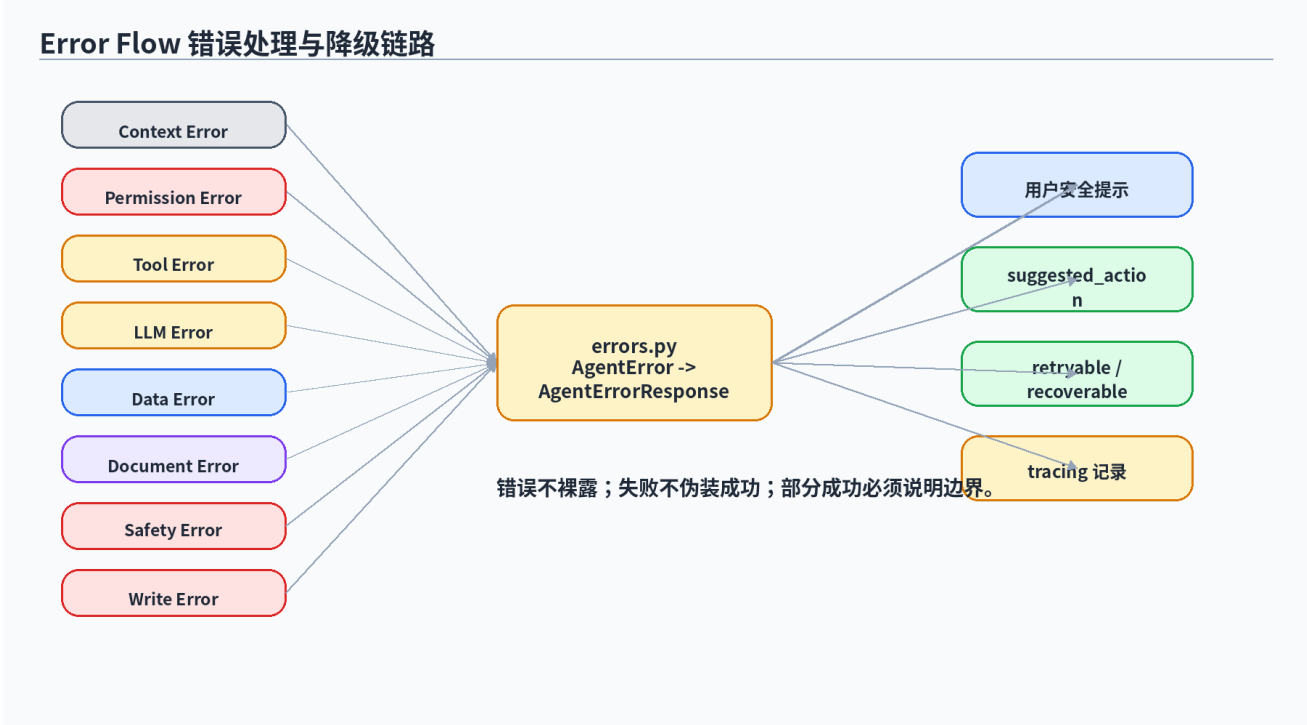


图 9：错误处理与降级链路

errors.py 将内部错误转成用户能理解、不会恐慌、不会泄露隐私、还能继续操作的回复。

错误类别	典型原因	用户提示策略
Context Error	缺少家庭上下文 / 目标成员不明确	请先选择家庭或家庭成员。
Permission Error	没有共享权限	只说没有权限，不透露数据是否存在。
Tool Error	工具超时 / 查询失败	说明暂时读取失败，可稍后重试；能部分回答就部分回答。
LLM Error	JSON 解析失败 / 输出为空	重试一次，仍失败则提示换种说法或手动记录。
Data Error	无记录 / 数据不足	系统内暂无记录不代表现实没有；提示补充记录。
Document Error	格式不支持 / OCR 失败 / 图片模糊	原件能保存先保存；提示重新上传或手动添加。
Safety Error	诊断 / 用药 / 停药请求	拒绝直接满足，转为整理资料和建议咨询医生。
Write Error	保存失败 / 部分保存成功	明确说明未保存或部分保存成功，不假装成功。

10.1 AgentErrorResponse

```
class AgentErrorResponse(BaseModel):
    success: bool = False
    error_code: str
    user_message: str
    recoverable: bool
    retryable: bool
```

```
suggested_action: str | None = None
trace_id: str | None = None
```

11. guards.py - 系统硬规则

guards.py 不做复杂业务判断，只执行绝不允许违反的红线。它像保险丝，防止 workflow 或 LLM 走偏。

硬规则	说明
没有 RuntimeContext，不执行 workflow	所有入口必须由 AgentService 创建 runtime。
没有 target_user_id，不查健康数据	避免 LLM 或用户模糊指代导致越权。
没有 current_family_id，不查家人数据	多家庭场景必须明确上下文。
没有 user_confirmed_action，不执行写入工具	AI 草稿不等于用户确认。
LLM 不得设置 current_user_id / target_user_id	身份和目标来自后端解析。
权限不足，不调用工具	不能先查再判断能不能展示。
无数据，不得说现实没有问题	系统内暂无记录不代表现实没有。
document 工具不得返回真实文件路径	只返回元信息、摘要、状态。
AI 提取结果不得直接入库	必须确认后保存。

12. AgentService / Runner / ToolCaller 接口骨架

12.1 AgentService

```
class AgentService:
    async def chat(self, user_id, family_id, message): ...
    async def create_free_text_record_draft(self, user_id, family_id, raw_text): ...
    async def save_free_text_record(self, user_id, family_id, draft, edits): ...
    async def extract_document(self, user_id, family_id, document_id, target_user_id): ...
    async def save_document_events(self, user_id, family_id, document_id, events): ...
    async def generate_daily_report(self, user_id, family_id, date): ...
    async def generate_doctor_visit_summary(self, user_id, family_id, message): ...
```

12.2 HarnessRunner

```
class HarnessRunner:
    async def run(self, runtime, workflow_callable, initial_state):
        trace = await tracing_manager.start_trace(runtime)
        try:
            guards.validate_runtime(runtime)
            memory_context = await memory_manager.build_memory_context(runtime=runtime,
workflow_name=runtime.workflow_name)
            initial_state["runtime"] = runtime
            initial_state["memory_context"] = memory_context
            initial_state["trace"] = trace
            result = await workflow_callable(initial_state)
            await tracing_manager.end_trace(trace, final_output=result.get("final_answer"),
status="success")
            return result
        except Exception as error:
            error_response = error_manager.to_user_response(error, runtime=runtime)
```

```

await tracing_manager.end_trace(trace, status="failed", error=error)
return error_response

```

12.3 HarnessToolCaller

```

class HarnessToolCaller:
    async def call_tool(self, runtime, tool_name, tool_input):
        spec = tool_registry.get_tool_spec(tool_name)
        guards.validate_tool_preconditions(runtime, spec)
        tool_registry.validate_tool_call(runtime=runtime, tool_name=tool_name)
        result = await tool_executor.execute(tool_name=tool_name, tool_input=tool_input,
runtime=runtime)
        guards.validate_tool_result(runtime, spec, result)
        await tracing_manager.record_tool_call(...)
        return result

```

13. Workflow 集成关系

Workflow	Harness 重点	主要工具
chat_workflow	runtime + member resolve + permissions + read tools + safety	get_recent_metrics, get_recent_blood_pressure, get_symptom_records, get_medical_events, get_medical_documents, get_latest_daily_report, get_active_alerts
free_text_record_workflow	草稿生成 + 用户确认 + 写入 guard	create_symptom_record_draft, save_symptom_record_from_draft
document_extract_workflow	先保存原件 + 文档权限 + 提取草稿 + 确认入档	extract_document_summary, save_medical_event_from_document
daily_report_workflow	系统白名单 + 规则算事实 + Safety + 保存日报	generate_daily_report 以及内部 service 数据收集
doctor_visit_summary_workflow	最高综合权限 + 数据来源说明 + 不诊断	get_recent_metrics, get_recent_blood_pressure, get_symptom_records, get_medical_events, get_active_alerts, 可选 get_medical_documents

13.1 对 LangGraph 的要求

- Workflow State 应包含 runtime、trace、memory_context。
- 所有 target_user_id 必须来自 RuntimeContext 或成员解析节点，不能来自 LLM 参数。
- 所有工具调用通过 HarnessToolCaller，不直接调用 tool function。
- 写入类 workflow 必须拆成“草稿阶段”和“确认保存阶段”。
- 回答最终节点必须调用 HarnessSafetyChecker。
- 错误必须抛 AgentError 或由 ErrorManager 统一转换。

14. V0 实施清单与测试清单

14.1 V0 必做模块

模块	V0 必做
runtime.py	RuntimeContext, RuntimeManager.create_context, with_target_member, mark_user_confirmed, validate_context
permissions.py	self access, family share_all, documents 独立检查, system whitelist, require_many
tool_registry.py	ToolSpec, TOOL_SPECS, validate_tool_call, can_expose_to_llm, 写入确认检查
safety.py	高风险症状检测、诊断/用药/保证语检测、边界提示与改写
tracing.py	start_trace, end_trace, record_tool_call, record_safety_check, 三张日志表
memory.py	agent_memories 表, user_preference 与 attention_focus, 写入前安全校验
errors.py	AgentError, AgentErrorResponse, ErrorManager.to_user_response, 标准错误码
guards.py	runtime guard, tool precondition guard, write confirmation guard, document path guard

14.2 关键测试用例

测试场景	预期结果
用户问自己的血压	本人 access 通过，调用血压查询工具，Safety 返回克制回答。
用户问爸爸血压但没有 family_id	Context Error，要求先选择家庭。
用户没有 documents 权限却查家人体检报告	PermissionDenied，不能透露资料是否存在。
LLM 尝试调用 save_symptom_record_from_draft	ToolRegistry 拦截，除非 user_confirmed_action = true。
用户描述胸痛和呼吸困难	Safety high_risk，返回及时就医提示，不诊断。
报告 OCR 失败	原始资料已保存，提示稍后重试或手动添加。
日报数据不足	生成 insufficient_data 或 skipped，不说身体没问题。
写入健康记录部分失败	明确说明哪部分已保存、哪部分失败。
工具超时	ErrorManager 给出可重试提示，Trace 记录 timeout。
Memory 准备写入“爸爸患有高血压”	Memory safety 拦截，改为关注重点或要求来源确认。

15. 下一步设计建议

Harness 文档定版后，下一步可以进入 Health Agent Core / Agent Engine 工程骨架设计。建议顺序：

25. AgentService 接口与 FastAPI 路由设计。
26. harness/ 目录代码骨架与 Pydantic schemas。
27. tools/ 执行器与 Business Services 对接。
28. workflows/ 的 LangGraph State 与节点落地。
29. 数据库 Alembic migration：agent_traces、agent_tool_calls、agent_safety_checks、agent_memories。
30. V0 端到端闭环：chat 查询血压、free_text_record 草稿确认保存、daily_report 生成。

最终总原则：没有 RuntimeContext 不执行；没有目标成员不查健康数据；没有权限不调用工具；没有用户确认不写入档案；没有 Safety 检查不返回健康回答；没有 Tracing 不上线核心 Agent 能力。